Narwhal 패밀리

📋 패밀리 개요

Narwhal은 2021년 Facebook (Meta)에서 제안한 **DAG 기반 Mempool + Consensus 분리 아키텍처**입니다. Mempool (데이터 가용성)과 Consensus (순서 결정)를 분리하여 높은 처리량을 달성합니다.

핵심 아이디어: Mempool과 Consensus 분리

```
┌──────────────────────────────────────────┐
│         Mempool Layer (Narwhal)          │  ← 트랜잭션 배치, DAG 구성
│                                          │
│  - Batch creation                        │
│  - DAG construction                      │
│  - Certificate formation                 │
│  - High throughput                       │
└──────────────────────────────────────────┘
                    ↓
         Certificates (ordered data)
                    ↓
┌──────────────────────────────────────────┐
│       Consensus Layer (Pluggable)        │  ← 순서 결정
│                                          │
│  - Tusk (leader-based)                   │
│  - Bullshark (leaderless)                │
│  - HotStuff (3-phase)                    │
│  - 다른 BFT 알고리즘도 가능            |
└──────────────────────────────────────────┘
```

패밀리 구성 (4가지)

1. **Narwhal (Original)**: DAG mempool + Leaderless ordering
2. **Narwhal-Tusk**: DAG mempool + Leader-based ordering
3. **Narwhal-Bullshark**: DAG mempool + Leaderless 3-round commit

4. **Narwhal-HotStuff**: DAG mempool + HotStuff consensus

공통 컴포넌트: DAG Mempool

DAG (Directed Acyclic Graph) 구조

```
        Round 0              Round 1              Round 2
Node 0: [Batch 0,0]──────▶[Batch 0,1]──────▶[Batch 0,2]
           |  ╲               |  ╲               |
           |   ╲              |   ╲              |
Node 1: [Batch 1,0]──────▶[Batch 1,1]──────▶[Batch 1,2]
           |  ╱ ╲             |  ╱ ╲             |
           | ╱   ╲            | ╱   ╲            |
Node 2: [Batch 2,0]──────▶[Batch 2,1]──────▶[Batch 2,2]
           |╱     ╲           |╱     ╲           |
          ╱|       ╲         ╱|       ╲          |
Node 3: [Batch 3,0]──────▶[Batch 3,1]──────▶[Batch 3,2]

각 Batch는 이전 라운드의 2f+1 Certificates를 참조
```

Batch와 Certificate

Batch:

```go
type Batch struct {
    nodeID       int
    round        int
    transactions []Transaction
    parents      []*Certificate  // 이전 라운드의 2f+1 certificates
}
```

Certificate:

```go
type Certificate struct {
    batchID    int
    signatures []Signature  // 2f+1 노드의 서명
}
```

Mempool 프로세스

```
각 노드가 병렬로:

1. Batch 생성:
   - 트랜잭션 수집
   - 이전 라운드 2f+1 certificates 참조
   - Batch 브로드캐스트

2. Vote 수집:
   - 다른 노드들의 Batch 수신
   - 검증 (부모 certificates 확인)
   - Vote 전송

3. Certificate 형성:
   - 2f+1 votes 수집
   - Certificate 생성
   - 다음 라운드로

4. DAG 구성:
   - Certificates가 DAG 노드
   - 부모 참조가 DAG 엣지
   - 모든 노드가 동일한 DAG 구축
```

DAG의 장점

✅ **병렬성**: 모든 노드가 동시에 Batch 제안
✅ **높은 처리량**: 네트워크 대역폭 최대 활용
✅ **분리된 관심사**: 데이터 전파 ≠ 순서 결정
✅ **모듈성**: 다른 Consensus 알고리즘 플러그인 가능

---

1. Narwhal (Original) - Leaderless

📋 개요

원본 Narwhal은 **Leaderless 방식**으로 DAG에서 자동으로 순서를 결정합니다.

Causal Ordering

DAG의 위상 정렬(Topological Sort)로 순서 결정:

```
┌─────┐   ┌─────┐   ┌─────┐
│ A   │→  │ C   │→  │ E   │
└─────┘   └─────┘   └─────┘
   ↓         ↑
┌─────┐   ┌─────┐
│ B   │→  │ D   │
└─────┘   └─────┘

Topological order: A, B, C, D, E
(또는 A, B, D, C, E - 여러 가능)
```

Round 진행

```go
func (node *NarwhalNode) advanceRound() {
    // 1. 이전 라운드에서 2f+1 certificates 수집
    if node.hasCertificates(node.round-1, 2f+1) {
        // 2. 다음 라운드로
        node.round++

        // 3. 새 Batch 생성
        node.createBatch()
    }
}
```

Commit Rule

```
Round R의 Certificate가 commit 가능:
  ⇔ Round R+k (k ≥ 2)에서 2f+1 certificates가
    R의 certificate를 (직간접적으로) 참조
```

코드 위치

```
internal/algorithms/narwhal/narwhal.go
```

---

2. Narwhal-Tusk - Leader-based Ordering

📋 개요

Narwhal-Tusk는 **리더가 명시적으로 순서를 제안**합니다. 원본 Tusk 논문의 구현입니다.

Ordering Proposal

```go
type OrderProposal struct {
    round            int
    leaderID         int
    certificates     []*Certificate  // 순서를 정할 certificates
    orderedBatchIDs  []int          // 제안된 순서
}
```

동작 과정

```
Round 1-5: Mempool layer (DAG 구성)
  - 모든 노드가 Batch 생성
  - Certificates 형성

Round 6: Ordering round
  Leader (round % n):
    1. Round 1-5의 certificates 수집
    2. OrderProposal 생성 (순서 제안)
    3. 모든 노드에게 전송

  Validators:
    1. OrderProposal 검증
    2. OrderVote 전송
    3. 2f+1 votes → OrderQC

  Leader:
    1. OrderQC 전송
    2. 모든 노드가 제안된 순서로 commit!
```

리더 선출

```go
func (node *NarwhalTuskNode) getLeader(round int) int {
    return (round + 1) % node.numNodes
}
```

순서 결정 알고리즘

```go
func (leader *NarwhalTuskNode) proposeOrder() {
    // 1. 미처리 certificates 수집
    certs := leader.collectCertificates()

    // 2. DAG 위상 정렬
    ordered := leader.topologicalSort(certs)

    // 3. OrderProposal 생성
    proposal := &OrderProposal{
        round:           leader.round,
        leaderID:        leader.id,
        certificates:    certs,
        orderedBatchIDs: ordered,
    }

    leader.broadcast(proposal)
}
```

Mempool + Consensus 시각화

```
Time ──────────────────────────────────────────▶

Mempool:  [R1] [R2] [R3] [R4] [R5] [R6] [R7]
           ↓    ↓    ↓    ↓    ↓    ↓    ↓
           Batch creation (continuous)

Ordering:        wait         [ORDER R1-5]
                              Leader proposes
                              ↓
                              [COMMIT]

Mempool:                      [R8] [R9] [R10]
                               ↓    ↓    ↓

Ordering:                            wait  [ORDER R6-10]
```

메시지 타입

```
ORDER_PROPOSAL:  Leader → All (ordering)
ORDER_VOTE:      All → Leader (vote)
ORDER_QC:        Leader → All (quorum certificate)
```

코드 위치

```
internal/algorithms/narwhal_tusk/narwhal_tusk.go
```

---

3. Narwhal-Bullshark - Leaderless 3-round

📋 개요

Narwhal-Bullshark는 **Leaderless 방식**으로 3라운드마다 자동으로 커밋합니다. Bullshark 논문의 핵심 아이디어입니다.

3-Round Commit Rule

```
┌───────┐   ┌───────┐   ┌───────┐
| R=0  |→ | R=1  |→ | R=2  |
|Certs |   |Certs |   |Certs |
└───────┘   └───────┘   └───────┘
                            ↓
                      Commit R=0!

┌───────┐   ┌───────┐   ┌───────┐
| R=1  |→ | R=2  |→ | R=3  |
|Certs |   |Certs |   |Certs |
└───────┘   └───────┘   └───────┘
                            ↓
                      Commit R=1!
```

규칙:

```
Round R의 Certificate 커밋:
  ⇔ Round R+2에서 2f+1 certificates 형성
```

Deterministic Ordering

모든 노드가 **동일한 순서**를 독립적으로 계산:

```go
func (node *NarwhalBullsharkNode) commitRound(round int) {
    // 1. Round R+2 도달 확인
    if node.currentRound < round + 2 {
        return
    }

    // 2. Round R의 certificates 수집
    certs := node.getCertificates(round)

    // 3. Deterministic 정렬 (모든 노드가 동일)
    ordered := sortByHash(certs)  // Hash 기준 정렬

    // 4. 순서대로 commit
    for _, cert := range ordered {
        node.commitBatch(cert.batch)
    }
}
```

Hash 기반 정렬

```go
func sortByHash(certs []*Certificate) []*Certificate {
    sort.Slice(certs, func(i, j int) bool {
        return certs[i].hash < certs[j].hash
    })
    return certs
}
```

모든 노드가 동일한 해시 값을 가지므로 동일한 순서!

Leaderless의 장점

✅ **리더 실패 없음**: 리더가 없으니 실패 걱정 없음
✅ **높은 parallelism**: 모든 노드가 독립적으로 동작
✅ **간단한 로직**: 3-round rule

동작 예제

```
Round 0:
  - Node 0: Batch A (cert hash: 0x123...)
  - Node 1: Batch B (cert hash: 0x456...)
  - Node 2: Batch C (cert hash: 0x789...)
  - Node 3: Batch D (cert hash: 0xabc...)

Round 1:
  - 각 노드가 새 Batch 생성 (R0 참조)

Round 2:
  - 각 노드가 새 Batch 생성 (R1 참조)
  - R2 도달! → R0 커밋
  - 모든 노드가 독립적으로 계산:
    - Hash 정렬: 0x123 < 0x456 < 0x789 < 0xabc
    - Order: A → B → C → D
  - 모든 노드가 동일한 순서로 커밋! ✓
```

코드 위치

```
internal/algorithms/narwhal_bullshark/narwhal.go
```

---

4. Narwhal-HotStuff - HotStuff Consensus

📋 개요

Narwhal-HotStuff는 **HotStuff 3-phase consensus**를 사용하여 certificates를 순서 지정합니다.

OrderBlock 구조

```go
type OrderBlock struct {
    blockID      int
    round        int
    certificates []*Certificate  // 순서를 정할 certificates
    parentQC     *QC             // 부모 블록의 QC
}
```

2-Layer 아키텍처

```
┌──────────────────────────────────────────┐
│        Mempool Layer (DAG)               │
│                                          │
│  Continuous batch creation               │
│  Certificate formation                   │
└──────────────────────────────────────────┘
                ↓ certificates
┌──────────────────────────────────────────┐
│    Consensus Layer (HotStuff)            │
│                                          │
│  OrderBlock 1:                           │
│    - Certs from Round 1-5                │
│    - Prepare → Precommit → Commit        │
│                                          │
│  OrderBlock 2:                           │
│    - Certs from Round 6-10               │
│    - Prepare → Precommit → Commit        │
└──────────────────────────────────────────┘
```

HotStuff Phases on OrderBlocks

```
Leader                  Replica1                Replica2
  |                        |                       |
  |--PREPARE(OrderBlock)-->|                       |
  |--PREPARE(OrderBlock)-->|---------------------->|
  |                        |                       |
  |<--VOTE-PREPARE---------|                       |
  |<--VOTE-PREPARE---------|<----------------------|
  |                        |                       |
  [prepareQC formed]       |                       |
  |                        |                       |
  |--PRECOMMIT+QC--------->|                       |
  |--PRECOMMIT+QC--------->|---------------------->|
  |                        |                       |
  |<--VOTE-PRECOMMIT-------|                       |
  |<--VOTE-PRECOMMIT-------|<----------------------|
  |                        |                       |
  [precommitQC formed]     |                       |
  |                        |                       |
  |--COMMIT+QC------------>|                       |
  |--COMMIT+QC------------>|---------------------->|
  |                        |                       |
  [OrderBlock committed → Order certificates!]     |
```

동작 과정

```go
// 1. Mempool layer (continuous)
for {
    batch := node.createBatch()
    cert := node.getCertificate(batch)
    node.addToDAG(cert)
}

// 2. Periodically create OrderBlock
if node.isLeader() && node.hasEnoughCertificates() {
    certs := node.collectUnorderedCertificates()
    orderBlock := &OrderBlock{
        certificates: certs,
        parentQC:     node.highQC,
    }
    node.hotstuffPropose(orderBlock)
}

// 3. HotStuff consensus on OrderBlock
node.runHotStuff(orderBlock)  // Prepare, Precommit, Commit

// 4. OrderBlock committed → Order certificates
if orderBlock.committed {
    for _, cert := range orderBlock.certificates {
        node.commitBatch(cert.batch)
    }
}
```

Mempool과 Consensus 타이밍

```
Timeline:

t=0-10s:   Mempool: 50 certificates formed
t=10s:     Consensus: OrderBlock 1 (certs 1-50)
t=10-12s:  HotStuff: Prepare → Precommit → Commit
t=12s:     Commit certs 1-50!

t=10-20s:  Mempool: 50 more certificates (51-100)
t=20s:     Consensus: OrderBlock 2 (certs 51-100)
t=20-22s:  HotStuff: Prepare → Precommit → Commit
t=22s:     Commit certs 51-100!
```

장점

✅ **HotStuff 안전성**: 증명된 BFT 합의
✅ **O(n) 메시지 복잡도**: HotStuff의 효율성
✅ **명확한 순서**: HotStuff가 결정
✅ **모듈성**: Mempool과 Consensus 완전 분리

코드 위치

```
internal/algorithms/narwhal_hotstuff/narwhal_hotstuff.go
```

📊 Narwhal 패밀리 비교

특징	Original	Tusk	Bullshark	HotStuff
Consensus	Leaderless	Leader-based	Leaderless	Leader-based
Ordering	Causal	Explicit	Deterministic	HotStuff
Commit Rule	k-round	2f+1 votes	3-round	3-phase
메시지 복잡도	$O(n^2)$	$O(n^2) + O(n)$	$O(n^2)$	$O(n^2) + O(n)$
리더 실패	N/A	View change	N/A	View change
Finality	~2 rounds	1 ordering	3 rounds	3 phases

주의: 모든 변형이 Mempool layer에서 $O(n^2)$ (모든 노드가 Batch 제안)

🎯 Mempool의 핵심 이점

1. 데이터 가용성 보장

```
전통적 BFT:
  Leader proposes block → All vote
  Problem: Leader가 블록 데이터 숨기면?
```

```
Narwhal:
  All nodes propose batches → Certificates
  Consensus는 certificates만 순서 지정
  데이터는 이미 모두가 가짐!
```

2. 높은 처리량

```
전통적 BFT:
  한 번에 하나의 블록 (리더 병목)

Narwhal Mempool:
  n개 노드가 동시에 n개 Batch 제안
  = n배 처리량!
```

3. Consensus 단순화

```
Consensus는 작은 certificates만 순서 지정
큰 트랜잭션 데이터는 이미 전파됨
```

💻 공통 구현 패턴

Certificate 검증

```go
func (node *NarwhalNode) validateCertificate(cert *Certificate) bool {
    // 1. 2f+1 signatures 확인
    if len(cert.signatures) < node.quorum {
        return false
    }

    // 2. 부모 certificates 확인
    for _, parent := range cert.batch.parents {
        if !node.hasCertificate(parent.batchID) {
            return false
        }
    }

    // 3. Round 확인
    if cert.batch.round != node.expectedRound() {
```

```go
        return false
    }

    return true
}
```

DAG 구축

```go
type DAGNode struct {
    certificate *Certificate
    parents     []*DAGNode
    children    []*DAGNode
}

func (node *NarwhalNode) addToDAG(cert *Certificate) {
    dagNode := &DAGNode{
        certificate: cert,
        parents:     node.getParents(cert),
    }

    // 부모의 children에 추가
    for _, parent := range dagNode.parents {
        parent.children = append(parent.children, dagNode)
    }

    node.dag[cert.batchID] = dagNode
}
```

🔬 성능 특성

장점

✅ **높은 처리량**: 모든 노드가 병렬로 Batch 제안
✅ **모듈성**: Mempool과 Consensus 분리
✅ **데이터 가용성**: DAG 구조로 보장
✅ **확장성**: 다양한 Consensus 알고리즘 플러그인

단점

❌ **복잡성**: 2-layer 아키텍처
❌ **메모리**: DAG 저장 필요
❌ **Finality 지연**: Mempool + Consensus 두 단계

🎓 학습 포인트

1. **Mempool과 Consensus 분리**:
    - 데이터 전파 ≠ 순서 결정
    - 각자 최적화 가능
2. **DAG의 역할**:
    - 병렬 Batch 제안
    - Causal 관계 유지
    - 데이터 가용성
3. **다양한 Ordering 전략**:
    - Leader-based (Tusk)
    - Leaderless (Bullshark)
    - BFT consensus (HotStuff)

🌟 실제 사용

- **Sui Blockchain**: Narwhal + Bullshark
- **Aptos**: Narwhal-like mempool
- **Celo**: Narwhal 연구 중

📚 참고 자료

- [Narwhal & Tusk 논문] Danezis et al. (2021)
- [Bullshark 논문] Spiegelman et al. (2022)
- [Sui Documentation] https://docs.sui.io/
- [코드] `internal/algorithms/narwhal*/`

← HotStuff Family | Autobahn Family →